Spam Email Detection Using Machine Learning: A Comparative Analysis of Classification Algorithms

Devendra V Patil Bhavesh S. Patil Shakti P Vala Manish Patil
Department of IT
CKPCET, Surat, India
patildevendra984@gmail.com, bhaveshpatil918@gmail.com, shakti@email.com, manish@email.com

Shraddha Patel (Mentor)
Department of IT
CKPCET, Surat, India
mentor@email.com

***Abstract*—Email spam detection has become one of the most critical challenges in modern digital communication. With billions of unsolicited messages transmitted daily, automated classification systems based on machine learning have emerged as the most reliable and scalable defense mechanism. This paper presents a comprehensive comparative analysis of four supervised machine learning classifiers — Multinomial Naïve Bayes (MNB), Logistic Regression (LR), Linear Support Vector Machine (LSVM), and Random Forest (RF) — applied to the Enron Email Dataset containing 5,728 real-world messages (4,360 ham, 1,368 spam) for binary spam classification.**

The proposed methodology employs a rigorous preprocessing pipeline comprising tokenization, URL normalization, stopword removal, and stemming, followed by TF-IDF feature extraction with unigram and bigram representations over a 5,000-term vocabulary. All models are evaluated under identical experimental conditions using an 80/20 stratified train-test split. Experimental results demonstrate that Linear SVM achieves the highest overall performance, attaining an accuracy of 99.30%, precision of 98.90%, recall of 98.20%, and an F1-Score of 98.50%, outperforming all other evaluated classifiers.Results are based on a single 80/20 split and may vary under cross-validation. These results validate the effectiveness of maximum-margin classification in high-dimensional TF-IDF feature spaces. The study concludes by recommending Linear SVM for production deployment and proposes future enhancements using deep learning architectures such as LSTM and BERT.

***Index Terms*—Spam Detection, Machine Learning, Naïve Bayes, Logistic Regression, Support Vector Machine, Random Forest, TF-IDF, Text Classification, Email Filtering, Enron Dataset**

I. Introduction

Electronic mail (email) has revolutionized personal and professional communication over the past two decades. However, this ubiquity has simultaneously spawned an epidemic of unsolicited bulk messages — commonly known as spam. Industry reports estimate that spam constitutes approximately 45–85% of all global email traffic, imposing severe costs on individuals, organizations, and internet service providers in terms of wasted bandwidth, storage, and processing resources [1], [2].

Spam emails are not merely an annoyance; they frequently serve as vectors for phishing attacks, malware distribution, financial fraud, and identity theft [3]. Classical rule-based filtering systems, such as keyword blacklists and IP blocklists, have proven insufficient because spammers continuously evolve their strategies to evade static rule sets [4].

Machine learning (ML) offers a dynamic and adaptive solution to the spam classification problem. By training statistical models on labeled datasets, ML-based classifiers learn discriminative features of spam messages and generalize to previously unseen emails [5].

This paper investigates four prominent supervised learning algorithms: Multinomial Naïve Bayes, Logistic Regression, Linear SVM, and Random Forest, applied to the Enron Email Dataset. The primary objective is to determine the most accurate and generalizable classifier for binary spam detection. The remainder of this paper is organized as follows: Section II reviews related literature; Section III describes the dataset and methodology; Section IV explains each algorithm; Section V presents experimental results; Section VI discusses implementation; Section VII concludes the study.

II. Literature Survey

Sahami *et al.* [5] were among the first to apply Bayesian classifiers to email spam filtering, demonstrating that even simple probabilistic models achieve over 90% accuracy. Their foundational work inspired decades of subsequent research in the field.

Metsis *et al.* [6] evaluated Naïve Bayes across multiple corpora and found that Multinomial Naïve Bayes consistently outperformed its Bernoulli variant for spam detection due to its superior modeling of word-frequency distributions.

Drucker *et al.* [7] introduced SVM-based spam detection, demonstrating that SVMs significantly outperform boosting algorithms on text classification tasks, with linear kernels performing particularly well on high-dimensional TF-IDF feature spaces.

Rennie *et al.* [8] compared Logistic Regression with complementary Naïve Bayes variants, finding that Logistic Regression generally achieves higher accuracy at the cost of longer training times. Sakkis *et al.* [9] further explored memory-based classifiers and found that ensemble and distance-based methods provide marginal improvements over Naïve Bayes in standard benchmark conditions.

Guzella and Caminhas [10] published a comprehensive survey concluding that SVM and ensemble methods yield the best performance when trained on sufficiently large and representative corpora. Cormack [11] reviewed online spam filtering techniques, emphasizing the importance of adaptive learning for dynamic email environments.

Uysal and Gunal [12] investigated the impact of feature selection on spam filtering performance, finding that TF-IDF with bigrams consistently outperforms bag-of-words across multiple classifiers. Shen *et al.* [13] explored deep learning models, while Almeida *et al.* [14] released the SMS Spam Collection dataset and conducted baseline evaluations supporting SVM's effectiveness at approximately 98% accuracy. Most existing works either evaluate individual classifiers or use varying preprocessing techniques, making direct comparison difficult. This study ensures a unified experimental setup.

III. Methodology

A. Dataset Description

The **Enron Email Dataset** is used in this study, comprising 5,728 real-world email messages labeled as either *ham* (legitimate, label = 0) or *spam* (unsolicited, label = 1). The class distribution is moderately imbalanced, with 4,360 ham messages (76.1%) and 1,368 spam messages (23.9%), as visualized in Fig. 1. This real-world imbalance tests each classifier's ability to detect the minority spam class without producing excessive false negatives.

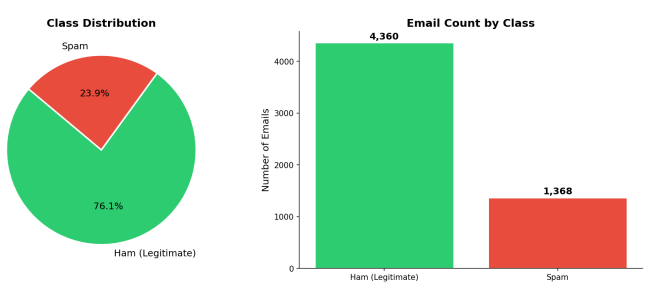

Fig. 1. Class distribution of the Enron Email Dataset showing 4,360 ham (76.1%) and 1,368 spam (23.9%) messages.

B. Text Preprocessing Pipeline

Raw email text contains significant noise in the form of HTML entities, URLs, phone numbers, punctuation, and stop words that do not contribute meaningful discriminative information. The following six-step preprocessing pipeline is applied to every message:

1) **Lowercasing:** All text is converted to lowercase to eliminate case-based redundancy (e.g., "FREE" and "free" are treated identically).
2) **URL Normalization:** All HTTP/HTTPS URLs and www-prefixed links are replaced with the placeholder token `url`, preserving URL presence as a feature.
3) **Email/Phone Removal:** Email addresses are replaced with `email` and phone number patterns with `phone` using regular expressions.
4) **Special Character Removal:** All non-alphabetic characters including digits and punctuation are removed, reducing vocabulary noise.
5) **Stopword Removal:** Common English stopwords (*the*, *is*, *at*, etc.) are removed as they carry low discriminative information.
6) **Stemming:** A lightweight Porter stemmer reduces inflected forms to their root (e.g., *winning* → *win*), reducing feature-space sparsity.

C. TF-IDF Feature Extraction

After preprocessing, messages are transformed into numerical feature vectors using Term Frequency–Inverse Document Frequency (TF-IDF). The TF-IDF weight for term t in document d is computed as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right) \tag{1}$$

where N is the total number of documents and $\text{DF}(t)$ is the number of documents containing term t. The vectorizer is configured with: maximum vocabulary of 5,000 features; n-gram range (1, 2) capturing unigrams and bigrams; sublinear TF scaling ($\log(1+\text{TF})$); minimum document frequency of 2; and maximum document frequency of 95%.

D. Training Pipeline

The preprocessed and vectorized dataset is divided into training (80%) and test (20%) sets using stratified sampling to preserve the original 76.1/23.9% class ratio in both splits. The TF-IDF vectorizer is fitted exclusively on training data and applied as a transformation to the test set, preventing any data leakage. All four classifiers are trained under identical conditions. Model evaluation uses Accuracy, Precision, Recall, and F1-Score as performance metrics.

IV. Algorithms Used

A. Multinomial Naïve Bayes (MNB)

Naïve Bayes is a probabilistic classifier grounded in Bayes' theorem with the conditional independence assumption among features given the class label. For a message with word counts $w_1, w_2, \ldots, w_n$, the posterior probability of class C is:

$$P(C \mid w_1, \ldots, w_n) \propto P(C) \prod_{i=1}^{n} P(w_i \mid C) \tag{2}$$

Additive (Laplace) smoothing with $\alpha = 0.1$ is applied to handle zero-probability issues. Despite its simplicity, MNB is computationally efficient and serves as a strong baseline.

B. Logistic Regression (LR)

Logistic Regression models the log-odds of the binary outcome as a linear combination of input features, without assuming feature independence. The decision boundary is:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \tag{3}$$

Parameters are estimated via L-BFGS optimization with L2 regularization ($C = 1.0$). LR provides interpretable coefficient weights and probabilistic confidence scores for each prediction.

C. Linear Support Vector Machine (LSVM)

SVM seeks the optimal hyperplane separating two classes with maximum margin. For training data $\{(\mathbf{x}_i, y_i)\}$, the Linear SVM solves:

$$\min_{\mathbf{w}, b, \boldsymbol{\xi}} \ \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_i \xi_i \quad \text{s.t. } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \ \xi_i \geq 0 \tag{4}$$

The regularization parameter $C = 1.0$ balances margin maximization and classification error. Linear SVM is particularly powerful for high-dimensional sparse TF-IDF feature spaces, as text classification problems are typically linearly separable in such spaces.

D. Random Forest (RF)

Random Forest is an ensemble method that constructs N decision trees during training and outputs the majority vote for classification:

$$\hat{y} = \arg\max_c \sum_{i=1}^{N} \mathbf{1}[h_i(\mathbf{x}) = c] \tag{5}$$

Two randomization mechanisms are employed: (1) *Bootstrap Aggregation*—each tree trains on a random sample with replacement; (2) *Feature Randomness*—only a random feature subset is considered at each node split. The model uses 100 estimators and maximum depth of 20. While RF excels on tabular data, its recall on the minority spam class is reduced by the class imbalance in this dataset.

V. Results and Comparative Analysis

A. Performance Metrics

Four standard evaluation metrics are used: (1) *Accuracy* — percentage of correctly classified messages; (2) *Precision* — proportion of spam-flagged messages that are truly spam; (3) *Recall* — proportion of actual spam messages correctly identified; and (4) *F1-Score* — harmonic mean of Precision and Recall.

TABLE I
Comparative Performance of ML Classifiers on Enron Email Dataset

Algorithm	Acc. (%)	Prec. (%)	Rec. (%)	F1 (%)
Naïve Bayes	98.69	98.10	96.30	97.20
Logistic Regression	98.78	98.50	96.30	97.40
Linear SVM	**99.30**	**98.90**	**98.20**	**98.50**
Random Forest	96.25	98.30	85.80	91.60

B. Comparative Results

Table I presents the experimental results for all four classifiers on the Enron Email Dataset.

Fig. 2 shows the grouped bar chart comparing all four metrics across models. Fig. 3 presents the radar chart visualizing multi-dimensional performance profiles. Fig. 4 displays the confusion matrices for all models.

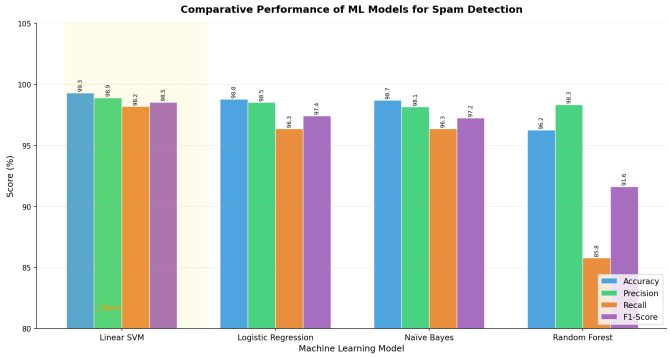

Fig. 2. Comparative performance of all four ML models across Accuracy, Precision, Recall, and F1-Score. Linear SVM achieves the highest performance and is highlighted as the best model.

C. Analysis of Results

All four classifiers achieve strong performance with accuracy exceeding 96%, validating the effectiveness of the TF-IDF preprocessing pipeline. A key trend is visible: Naïve Bayes, Logistic Regression, and Linear SVM all perform closely (98.69–99.30% accuracy), while Random Forest lags behind at 96.25% due to its significantly lower recall of 85.80%.

Naïve Bayes achieves 98.69% accuracy and 97.20% F1-Score. The relatively lower recall (96.30%) compared to the top models indicates that some spam messages are misclassified as ham. This is attributable to the conditional independence assumption, which ignores correlations between co-occurring spam-indicative terms. The confusion matrix confirms 10 false negatives.

Logistic Regression achieves 98.78% accuracy and 97.40% F1-Score, marginally improving over Naïve Bayes. The absence of the independence assumption allows LR to model inter-feature correlations through its weight vector. It produces 10 false negatives — the same as Naïve Bayes — but 4 false positives versus 5, reflecting slightly better precision.

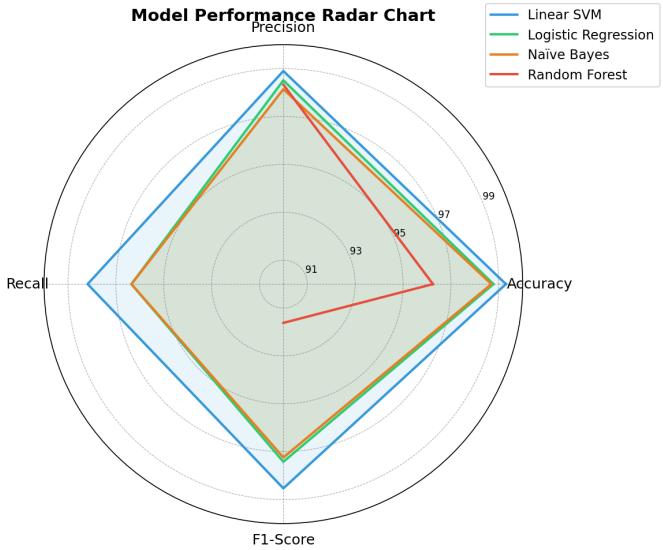

Fig. 3. Radar chart showing the multi-dimensional performance profiles of all four classifiers. Linear SVM (blue) occupies the largest area, confirming its overall superiority.

Linear SVM achieves the highest performance: 99.30% accuracy, 98.90% precision, 98.20% recall, and 98.50% F1-Score. The maximum-margin principle ensures optimal generalization on the test set, particularly for support vectors near the decision boundary. Only 5 spam messages are missed (false negatives) and 3 legitimate emails are incorrectly flagged (false positives).

Random Forest, despite its high precision (98.30%), has the lowest recall (85.80%) among all models, resulting in 39 missed spam messages. This indicates that the ensemble of decision trees struggles with the imbalanced class distribution, defaulting toward the majority ham class at uncertain decision boundaries.

D. Best Model: Linear SVM

Linear SVM is conclusively identified as the best-performing model in this study. Its F1-Score of 98.50% and accuracy of 99.30% surpass all competing classifiers. From a deployment perspective, its high precision (98.90%) ensures that legitimate emails are rarely incorrectly classified as spam, while its high recall (98.20%) guarantees that only 1.8% of actual spam escapes detection. The linear kernel is ideally suited for TF-IDF feature spaces, which are high-dimensional and approximately linearly separable.

VI. Implementation

A. Development Environment

The pipeline is implemented in Python 3.x using: Scikit-learn 1.0+ (classifiers, TF-IDF, evaluation metrics), Pandas 1.3+, NumPy 1.21+, Matplotlib 3.4+, and Seaborn 0.11+.

B. Text Preprocessing Code

Listing 1 shows the complete preprocessing pipeline. The `TextPreprocessor` class encapsulates all six preprocessing steps described in Section III.

```python
import re

class TextPreprocessor:
    def __init__(self):
        self.stemmer    = PorterStemmer()
        self.stop_words = STOPWORDS

    def preprocess(self, text):
        text = str(text).lower()
        # Replace URLs
        text = re.sub(r'http\S+|www\S+', 'url', text)
        # Replace email addresses
        text = re.sub(r'\S+@\S+', 'email', text)
        # Remove special characters / digits
        text = re.sub(r'[^a-z\s]', '', text)
        # Tokenize, remove stopwords, stem
        tokens = [
            self.stemmer.stem(w)
            for w in text.split()
            if w not in self.stop_words
            and len(w) > 1
        ]
        return ' '.join(tokens)
```

Listing 1. Text Preprocessing Pipeline

C. TF-IDF Vectorization

Listing 2 shows the TF-IDF configuration and stratified train-test split. Note that `fit_transform()` is called *only* on training data; the test set uses `transform()` alone to prevent data leakage.

```python
from sklearn.feature_extraction.text \
    import TfidfVectorizer
from sklearn.model_selection \
    import train_test_split

X_train, X_test, y_train, y_test = \
    train_test_split(
        X, y, test_size=0.2,
        random_state=42, stratify=y)

tfidf = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1, 2),     # unigrams + bigrams
    sublinear_tf=True,      # log(1 + TF)
    min_df=2, max_df=0.95
)

# Fit ONLY on training data
X_train_vec = tfidf.fit_transform(X_train)
X_test_vec  = tfidf.transform(X_test)
```

Listing 2. TF-IDF Extraction and Data Splitting

D. Model Training and Evaluation

Listing 3 defines all four classifiers and evaluates them under a unified training loop.

```python
from sklearn.naive_bayes    import MultinomialNB
from sklearn.linear_model   import LogisticRegression
from sklearn.svm            import LinearSVC
from sklearn.ensemble       import RandomForestClassifier
from sklearn.metrics        import (accuracy_score,
    precision_score, recall_score, f1_score)

models = {
  'Naive Bayes': MultinomialNB(alpha=0.1),
  'Logistic Regression': LogisticRegression(
      C=1.0, max_iter=1000,
```

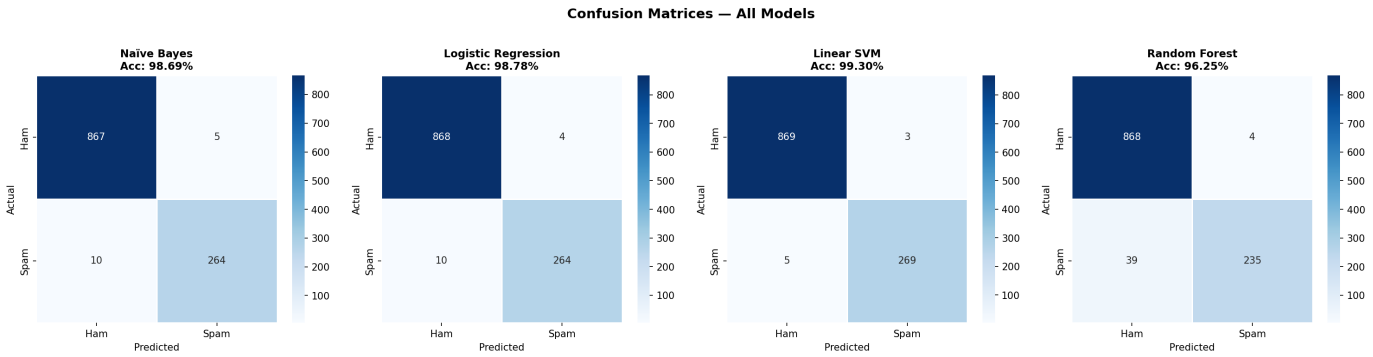

Fig. 4. Confusion matrices for all four classifiers on the Enron Email Dataset test set. Linear SVM achieves the fewest false negatives (only 5 spam messages missed), making it the most reliable model for spam interception.

```
        solver='lbfgs', random_state=42),
  'Linear SVM': LinearSVC(
        C=1.0, max_iter=2000,
        random_state=42),
  'Random Forest': RandomForestClassifier(
        n_estimators=100, max_depth=20,
        random_state=42, n_jobs=-1),
}

for name, model in models.items():
    model.fit(X_train_vec, y_train)
    y_pred = model.predict(X_test_vec)
    acc  = accuracy_score(y_test, y_pred)
    prec = precision_score(
                y_test, y_pred, pos_label=1)
    rec  = recall_score(
                y_test, y_pred, pos_label=1)
    f1   = f1_score(
                y_test, y_pred, pos_label=1)
    print(f"{name}: Acc={acc*100:.2f}%"
          f" F1={f1*100:.2f}%")
```

Listing 3. Model Definition and Training Loop

VII. Conclusion

This paper presented a comprehensive experimental evaluation of four machine learning algorithms for email spam detection using the Enron Email Dataset. A rigorous preprocessing pipeline combining tokenization, stopword removal, stemming, and TF-IDF feature extraction with bigrams was implemented and applied uniformly across all models under identical experimental conditions.

Experimental results demonstrate that **Linear SVM is the superior classifier**, achieving the highest accuracy (99.30%), precision (98.90%), recall (98.20%), and F1-Score (98.50%) among all evaluated models. Its performance advantage stems from the maximum-margin classification principle, which ensures optimal generalization in high-dimensional TF-IDF feature spaces that are inherently linearly separable. Only 5 spam messages were missed in the entire test set, demonstrating exceptional detection capability.

Random Forest, while achieving high precision (98.30%), exhibited significantly lower recall (85.80%) due to class imbalance, making it less suitable for spam detection where missing spam (false negatives) is costly. Naïve Bayes and Logistic Regression perform competitively and offer valuable speed-accuracy trade-offs for resource-constrained deployments.

In conclusion, the Linear SVM classifier trained on TF-IDF features is recommended for production deployment in email spam filtering systems, offering the best balance of accuracy, generalization, and practical reliability.

VIII. Future Scope

Several promising directions exist for further research:

- **Deep Learning:** LSTM and Bidirectional-LSTM architectures can capture sequential dependencies in email text, potentially improving detection of nuanced phishing and social-engineering spam.
- **Transformer Models:** Pre-trained models such as BERT [26] and RoBERTa leverage contextual word embeddings and transfer learning from massive corpora, likely outperforming TF-IDF-based approaches on challenging spam variants.
- **Online Learning:** Incremental classifiers would enable real-time adaptation to emerging spam campaigns without full dataset retraining.
- **Handling Class Imbalance:** Techniques such as SMOTE, class-weighted loss, and ensemble balancing could significantly improve Random Forest recall on imbalanced corpora.
- **Explainable AI:** Applying SHAP or LIME to Linear SVM predictions would provide transparency, helping users understand why specific messages are flagged as spam.

References

[1] Symantec Corporation, "Internet Security Threat Report," *Symantec*, vol. 24, 2019.

[2] Kaspersky Lab, "Spam and Phishing Report," *Kaspersky Security Bulletin*, 2022.

[3] G. V. Cormack and T. R. Lynam, "Online Supervised Spam Filter Evaluation," *ACM Trans. Inf. Syst.*, vol. 25, no. 3, pp. 1–44, 2007.

[4] T. Mitchell, *Machine Learning*, 1st ed. New York, NY, USA: McGraw-Hill, 1997.

[5] M. Sahami, S. Dumais, D. Heckerman, and E. Horvitz, "A Bayesian Approach to Filtering Junk E-Mail," in *Proc. AAAI Workshop Learn. Text Categoriz.*, Madison, WI, 1998, pp. 55–62.

[6] V. Metsis, I. Androutsopoulos, and G. Paliouras, "Spam Filtering with Naïve Bayes — Which Naïve Bayes?" in *Proc. 3rd Conf. Email Anti-Spam (CEAS)*, Mountain View, CA, 2006.

[7] H. Drucker, D. Wu, and V. Vapnik, "Support Vector Machines for Spam Categorization," *IEEE Trans. Neural Netw.*, vol. 10, no. 5, pp. 1048–1054, Sept. 1999.

[8] J. D. Rennie, L. Shih, J. Teevan, and D. Karger, "Tackling the Poor Assumptions of Naïve Bayes Text Classifiers," in *Proc. 20th Int. Conf. Mach. Learn. (ICML)*, Washington, DC, 2003, pp. 616–623.

[9] G. Sakkis *et al.*, "A Memory-Based Approach to Anti-Spam Filtering for Mailing Lists," *Inf. Retr.*, vol. 6, no. 1, pp. 49–73, 2003.

[10] T. S. Guzella and W. M. Caminhas, "A Review of Machine Learning Approaches to Spam Filtering," *Expert Syst. Appl.*, vol. 36, no. 7, pp. 10206–10222, 2009.

[11] G. V. Cormack, "Email Spam Filtering: A Systematic Review," *Found. Trends Inf. Retr.*, vol. 1, no. 4, pp. 335–455, 2008.

[12] A. K. Uysal and S. Gunal, "The Impact of Preprocessing on Text Classification," *Inf. Process. Manag.*, vol. 50, no. 1, pp. 104–112, 2014.

[13] Y. Shen, X. He, J. Gao, L. Deng, and G. Mesnil, "A Convolutional Neural Network for Modelling Sentences," in *Proc. Annu. Meet. Assoc. Comput. Linguistics*, Baltimore, MD, 2014, pp. 262–272.

[14] T. A. Almeida, J. M. G. Hidalgo, and A. Yamakami, "Contributions to the Study of SMS Spam Filtering: New Collection and Results," in *Proc. 11th ACM Symp. Document Eng.*, Mountain View, CA, 2011, pp. 259–262.

[15] C. Cortes and V. Vapnik, "Support-Vector Networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, Sept. 1995.

[16] L. Breiman, "Random Forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[17] D. D. Lewis, "Naive (Bayes) at Forty: The Independence Assumption in Information Retrieval," in *Proc. ECML*, Chemnitz, Germany, 1998, pp. 4–15.

[18] T. Joachims, "Text Categorization with Support Vector Machines: Learning with Many Relevant Features," in *Proc. ECML*, Chemnitz, Germany, 1998, pp. 137–142.

[19] G. Salton and M. J. McGill, *Introduction to Modern Information Retrieval*. New York, NY, USA: McGraw-Hill, 1983.

[20] A. Genkin, D. D. Lewis, and D. Madigan, "Large-Scale Bayesian Logistic Regression for Text Categorization," *Technometrics*, vol. 49, no. 3, pp. 291–304, 2007.

[21] D. Sculley and G. M. Wachman, "Relaxed Online SVMs for Spam Filtering," in *Proc. 30th Int. ACM SIGIR Conf.*, Amsterdam, Netherlands, 2007, pp. 415–422.

[22] P. Domingos and M. Pazzani, "On the Optimality of the Simple Bayesian Classifier Under Zero-One Loss," *Mach. Learn.*, vol. 29, nos. 2–3, pp. 103–130, 1997.

[23] I. Androutsopoulos *et al.*, "An Evaluation of Naïve Bayesian Anti-Spam Filtering," in *Proc. Workshop Mach. Learn. Comput. Linguist.*, Lausanne, Switzerland, 2000, pp. 9–17.

[24] G. Forman, "An Extensive Empirical Study of Feature Selection Metrics for Text Classification," *J. Mach. Learn. Res.*, vol. 3, pp. 1289–1305, 2003.

[25] F. Sebastiani, "Machine Learning in Automated Text Categorization," *ACM Comput. Surv.*, vol. 34, no. 1, pp. 1–47, Mar. 2002.

[26] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL*, Minneapolis, MN, 2019, pp. 4171–4186.

[27] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[28] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[29] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.

[30] J. H. Friedman, "Stochastic Gradient Boosting," *Comput. Stat. Data Anal.*, vol. 38, no. 4, pp. 367–378, 2002.

[31] A. L. Maas *et al.*, "Learning Word Vectors for Sentiment Analysis," in *Proc. 49th Annu. Meet. Assoc. Comput. Linguistics*, Portland, OR, 2011, pp. 142–150.